EdgeEngine: A Thermal-Aware Optimization Framework for Edge Inference

Amirhossein Ahmadi, Hazem A. Abdelhafez, Karthik Pattabiraman and Matei Ripeanu
{amirhmi,hazem,karthikp,matei}@ece.ubc.ca
University of British Columbia

ABSTRACT

Heterogeneous edge platforms enable the efficient execution of machine learning inference applications. These applications often have a critical constraint (such as meeting a deadline) and an optimization goal (such as minimizing energy consumption). To navigate this space, existing optimization frameworks adjust the platform's frequency configuration for the CPU, the GPU and/or the memory controller. However, existing optimization frameworks have two limitations. First, edge applications are frequently deployed in environments where they are exposed to ambient temperature variations. Second, a recent study has shown that temperature has a significant impact on edge platform characteristics. In this context, today's frequency optimization frameworks (which are *thermal-oblivious*) select frequency configurations that either violate the application's constraints, or are sub-optimal in terms of the optimization goal.

To address these shortcomings, we propose *EdgeEngine*, a lightweight *thermal-aware* optimization framework. EdgeEngine monitors the platform's temperature and uses reinforcement learning to adjust the frequency configuration of all underlying platform resources to meet the application's constraints. We find that EdgeEngine meets the application's constraint, and achieves *29%* lower energy consumption (up to 2x) and *41%* fewer violations compared to existing state-of-the-art thermal-oblivious optimization frameworks.

CCS CONCEPTS

• **Computer systems organization → Embedded systems**.

KEYWORDS

Performance and power variation, Edge computing, Reinforcement Learning

ACM Reference Format:
Amirhossein Ahmadi, Hazem A. Abdelhafez, Karthik Pattabiraman and Matei Ripeanu. 2023. EdgeEngine: A Thermal-Aware Optimization Framework for Edge Inference. In *The Eighth ACM/IEEE Symposium on Edge Computing (SEC '23), December 6–9, 2023, Wilmington, DE, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3583740.3626616

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SEC '23, December 6–9, 2023, Wilmington, DE, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0123-8/23/12…$15.00
https://doi.org/10.1145/3583740.3626616

1 INTRODUCTION

Context. Recent advances in machine learning (ML) research have led to numerous applications (e.g., computer vision, autonomous driving) that are deployed on resource-constrained edge platforms (e.g., drones, autonomous vehicles, IoT devices) [19, 33]. These applications usually have specific Quality of Service (QoS) objectives (e.g., latency, energy budget, accuracy) for the target inference tasks. Violating these objectives can have severe consequences [14], e.g., an autonomous vehicle colliding with an obstacle due to missing an object detection deadline, a drone crashing after running out of battery.

At the same time, the advent of heterogeneous embedded edge platforms with a System on Chip (SoC) design that integrate multiple processing units (e.g., CPU, GPU) on the same chip, such as the NVIDIA Jetson platform family, has enabled the execution of ML inference workloads at the edge [14, 16]. These platforms are designed to speed up ML-related tasks in a power-efficient manner, and employ tunable processing units and memory components [3, 5].

Frequency optimization frameworks [7, 8, 10, 11, 14, 17, 31] (§3.3) have the important role of adjusting the frequency configuration of the underlying components of the edge platform aiming to satisfy application constraints while simultaneously optimizing one or more unconstrained dimensions, e.g., meet a target deadline while minimizing energy consumption.

Problem Statement. A significant challenge for such optimization frameworks is that edge devices are often deployed in uncontrolled environments with high temperature variations. For example, temperatures in the desert can vary from below 0°C at night to 70°C when exposed directly to the sun [13]. Our work (summarized in §2, [6]) has found that there is significant variation in edge platforms' power consumption (up to 40%) and running time (to 5%) when the ambient temperature changes from 25°C to 70°C. Unfortunately, existing frequency optimization frameworks are *thermal-oblivious*, i.e., their design does not consider the ambient temperature. As a result, these frameworks either provide sub-optimal solutions or, more importantly, fail to meet their QoS objectives (§3.2 and §3.3). To tackle the challenge posed by ambient temperature fluctuations, optimization frameworks must adapt in real-time. The adaptation should be: *(i) swift*, ensuring timely adjustments to changing temperatures, and *(ii) conservative*, minimizing the number of violations during the adaptation.

However, the large frequency configuration space of current heterogeneous edge platforms poses a challenge for achieving real-time adaptation, and identifying efficient configurations. This is influenced by two factors. First, these platforms have numerous components that are frequency configurable, namely the processing units (e.g., CPU, GPU, application-specific accelerators) as well

as the memory controller. Second, each component offers a wide range of configurable frequencies, often spanning over a 10x range with dozens of available levels. This leads to an overall configuration space with tens of thousands of unique configurations, where the impact of the frequency configuration choice on application execution time or power consumption leads to a range spanning two orders of magnitude (§3.1).

Objectives. To address this issue, this study explores the design of a practical *thermal-aware* frequency optimization framework for edge inference that aims to satisfy the following six objectives: *(O_1) Adaptation to platform temperature:* ability to adjust the system's frequency configuration based on the current platform temperature; *(O_2) Cover the full configuration space:* able to select energy/performance efficient configuration by adjusting frequencies for *all* the underlying components (e.g., CPU, GPU, memory controller); *(O_3) Real-time adaptation*: ability to swiftly adapt with a low number of violations during the adaptation process; *(O_4) Robust to noise:* ability to handle the inherent variability in execution time and power consumption during application execution; *(O_5) Lightweight design:* low resource overheads such that it can be deployed on resource-constrained edge devices; and *(O_6) Generalizability:* ability to be deployed in different environments and support a large class of inference applications.

Approach. To this end, we propose *EdgeEngine*[1], a lightweight *thermal-aware* optimization framework for real-time inference on heterogeneous edge devices. EdgeEngine configures the platform's CGM[2] frequency configuration to efficiently satisfy the applications' constraints and optimization objectives in response to variations in platform temperature.

In EdgeEngine, users specify the application's constraints in the *energy* or *time* dimensions (i.e., an energy budget or target deadline) and an optimization objective. At execution time, EdgeEngine continuously observes the platform's temperature and the application behavior, and selects a CGM frequency configuration to satisfy the constraint while optimizing in the unconstrained dimension.

EdgeEngine uses *Reinforcement Learning (RL)* to adapt to the changes in edge device characteristics induced by platform temperature variations (§4). EdgeEngine employs two domain-specific heuristics that enable real-time adaptation when a temperature is observed for the first time. These heuristics accelerate the adaptation process and minimize constraint violations. First, EdgeEngine can identify and discard configurations that have a high probability of violating constraints, thus preventing their selection. Second, whenever EdgeEngine detects a constraint violation, it chooses a frequency configuration that leads to efficient space exploration while avoiding future violations.

Contributions. *To the best of our knowledge, EdgeEngine is the first thermal-aware frequency optimization framework for edge inference.* EdgeEngine incorporates real-time adaptation, and adjusts the frequency configuration of *all* the underlying components in response to platform temperature changes during execution, without relying on prior knowledge. Our contributions are:

- *Highlighting the effect of thermal-obliviousness on existing edge inference frequency optimizers (§3.2).* We analyze the effect of ambient temperature changes on energy-constrained applications in the absence of thermal adaptation (§3.2 and §3.3). We find that, depending on the framework and evaluation scenario, temperature changes can lead to high rates of energy constraint violations and up to 20% higher execution time. To conduct this study, we build on prior work that reveals that ambient temperature variation can lead to significant differences in power consumption (up to 40%), and non-negligible differences in the execution time (up to 5%) of edge inference applications (summarized in (§2).
- *Proposing, EdgeEngine, a thermal-aware optimization framework based on RL (§4).* EdgeEngine supports both deadline- and energy-constrained applications (§5.5). EdgeEngine incorporates thermal awareness and leverages RL and several domain-specific heuristics to enable rapid adaptation with minimal violations in response to temperature changes. Importantly, EdgeEngine is capable of adjusting the frequency for all the underlying components to identify the most efficient settings. Finally, EdgeEngine is robust to variability during inference execution, ensuring compliance with the application's constraints (§6.2).
- *Evaluating EdgeEngine on various workloads, constraints, and environments as well as comparing it to related work (§6).* We design and implement an emulation environment (§5.4 to systematically expose the platform to different ambient temperatures, while running various ML inference workloads with both *loose and tight energy and deadline constraints* (§5.5). We compare EdgeEngine with two state-of-art optimization frameworks *NeuOS* [7] and *AutoScale* [17], with *TOF* (a thermal-oblivious variant of EdgeEngine), as well as with a perfect-knowledge *ORACLE* (that always selects the optimal configuration) in terms of *(i)* satisfying the application constraint, *(ii)* optimization goal - energy efficiency (energy per inference) for deadline-constrained applications, and runtime (i.e., inference execution time) for energy-constrained applications.

  We demonstrate that EdgeEngine adapts successfully in settings that have energy or runtime constraints. When compared to existing state-of-the-art thermal-oblivious optimization frameworks, EdgeEngine demonstrates significant improvements. It achieves an average of **29%** lower energy consumption and **26%** fewer violations for deadline-constrained applications, as well as an average of **8%** lower runtime and **41%** fewer violations for energy-constrained applications. Notably, EdgeEngine's selected configurations incur only a slight increase of up to **3%** in runtime and energy consumption compared to those selected by the perfect knowledge ORACLE (and lead to no violations).

  Finally, the lightweight version of reinforcement learning used (Q-Learning) leads to low performance and memory overheads thus making EdgeEngine well-suited for deployment on computationally-constrained embedded devices.

2 THE IMPACT OF AMBIENT TEMPERATURE

Our prior work [6] characterized the impact of ambient temperature on the runtime and power consumption of machine learning inference tasks on Nvidia Jetson TX2 heterogeneous edge devices.

[1]Available at: https://github.com/DependableSystemsLab/EdgeEngine
[2]A CGM frequency configuration is a unique combination of **C**PU, **G**PU, and **M**emory frequencies.

This section briefly presents the methodology and the characterization results. Additional details on the methodology are available in our study characterizing variability among nominally identical heterogeneous edge boards [4]

2.1 Methodology

Temperature-Controlled Chamber. We place the Edge platform inside a temperature-controlled chamber (similar to [2]), which can control the temperature and humidity conditions experienced by the board during its operation. We systematically expose the board to different ambient temperatures.

Environmental Setup. To create a controlled and stable environment for accurate data collection, the environmental setup involves various steps to reduce experiment bias [23] (i.e., noise) including: *(i)* maintaining the fan on all the time, *(ii)* CPU core isolation and thread affinity, *(iii)* disabling concurrent user-space applications, *(iv)* preventing I/O operations and remote access, and *(v)* enabling PyTorch-specific optimizations at all times. We also make sure that hardware throttling does not kick in.

Data Collection. To collect runtime and power data, we follow a two-step procedure. First, we reboot the board, and run the target benchmark five times to avoid cold start effects at both the application and board levels. Second, we repeatedly perform a sequence that involves: *(i)* sending input from the CPU to the GPU, *(ii)* running the benchmark on the GPU, and *(iii)* sending the result back. We refer to this sequence as a *block*. We use the *block*, rather than a single benchmark execution, to eliminate the overhead associated with the CUDA timers and synchronization. We determine the size of each block based on two criteria: *(i)* it must have at least 100 iterations, and *(ii)* the CUDA runtime and synchronization overhead should be less than 0.1% of the block runtime.

Terminology. We define a *runtime observation* as the average benchmark execution time during a block, and a *power observation* as the average power readings of onboard power monitors during the block execution. We refer to a collection of observations as a *sample*.

Non-Parametric Statistical Techniques. We ran Anderson-Darling normality test [12] on runtime and power samples, and found that the data does not follow a normal distribution. As a consequence, we employ non-parametric statistical analysis. We determine the stopping point for data collection based on observed values (similar to [20]) such that our estimation of the median is robust. Specifically, we define two stop criteria for collecting observations for a *sample*: *(i)* the sample must contain at least 50 observations, and *(ii)* the Relative Margin of Error (RME) for the estimation of the median should be $\leq 0.5\%$ with a confidence level of 99%.

2.2 Impact on Power Consumption

Figure 1 presents power consumption for each workload in the high (top plot) and medium (bottom plot) frequency settings at both low (left, grey boxplots) and high (right) temperatures. In the high-frequency configuration, the average increase of the median power consumption across all workloads is **18.4%** at 70°C compared to 25°C, with the Shufflenetv2 exhibiting the highest increase: **27.1%**. In the medium frequency setting, the average increase is even higher: **21.6%**, with Shufflenetv2 again demonstrating the maximum increase: **38.1%**.

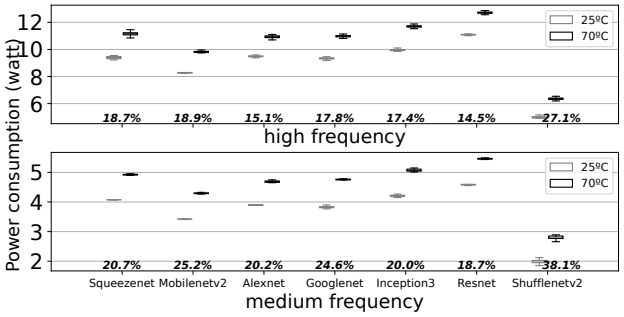


Figure 1: Boxplots present the distribution of measured power while executing different CNN inference workloads at 25 (left, grey boxplots) and 70°C (right) in the high (top plot) and medium (bottom) frequency configurations. Boxplots present the median, top and bottom quartiles, and 5th/95th %-tiles (shown by whiskers). Outliers were omitted for clarity. For each model, the differences (as %-tiles) between the medians are shown on the x-axis. Lower values are better.

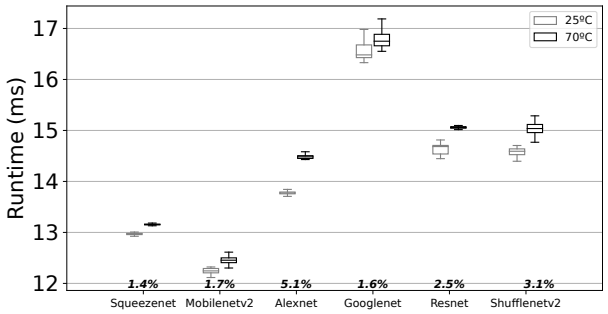


Figure 2: Boxplots present the distribution of the medians of the collected runtime samples for different CNN inference workloads at 25 (left, grey boxplots) and 70°C (right) in the high frequency configurations. Boxplots have the same meaning as in Figure 1. For each model, the differences (%) between the medians at different temperatures are shown on the x-axis. Lower values are better.

2.3 Impact on Runtime

Due to the time-consuming nature of collecting runtime data, particularly in the medium frequency configuration, we focus solely on the high frequency configuration[3]. Figure 2 indicates an increase in median runtime at 70°C compared to 25°C for all models. On average, there is a **2.5%** increase in median runtime, with a maximum of **5.1%** observed for Alexnet. For visual clarity, Inception3 is omitted from the plot, but it also shows a similar increase (2.1%) in median runtime.

[3]The data collected for only a few benchmarks in the medium frequency configuration supports the conclusion we present here.

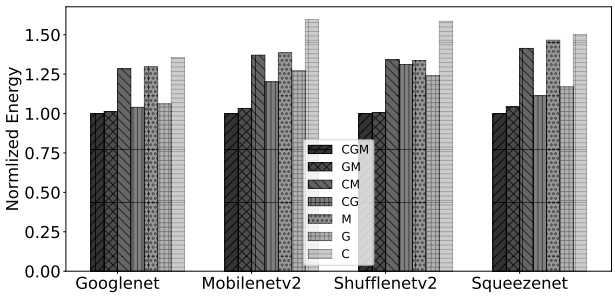


Figure 3: The impact of restricting the configuration space. The plot presents the energy consumption of various ML inference workloads choosing the best frequency configuration that meets a 50FPS (i.e., 20 ms deadline) QoS constraint and optimizes for energy. Different bars consider a different (restricted) configuration space – the components tuned for each bar are represented by 'C' for CPU, 'G' for GPU, and 'M' for the Memory controller. Data is normalized by the (best) CGM combination. (Lower is better.)

3 THERMAL-OBLIVIOUS OPTIMIZATION FRAMEWORKS

Frequency optimization frameworks [8, 10, 11, 14] control the device's frequency configuration and work on a two-dimensional constraint space: *time* and *energy*. For a given workload, the framework will adjust the frequency configuration for some/all components of the device (§3.1), to find a configuration that: *(i)* satisfies the critical constraint dimension, and *(ii)* is efficient for the other unconstrained dimension (e.g., for a deadline-constrained application, the frequency configuration must both meet the target deadline and be energy efficient).

This section highlights the importance of taking into account the effects of temperature on power and runtime in the context of a frequency optimization framework (§3.2), and discusses the limitations of existing *thermal-oblivious* frameworks in achieving the objectives highlighted in §1. All experiments use the experimental setup described later in §5. The section starts by describing the characteristics of the frequency configuration space (§3.1) and concludes with a brief introduction to RL, the key technique we use to support *thermal-awareness* (§4).

3.1 The Frequency Configuration Space

Today's edge systems have highly configurable processing units and memory controllers. Nvidia Jetson TX2, for example, consists of four big Nvidia Denver cores, two little ARM cores, 256-core Nvidia Pascal GPU, and 8 GB of shared memory. Each of these components has voltage and frequency gates that can be configured using the DVFS software module [7].

The resulting frequency configuration space is:

- *large:* with over 2,000 unique configurations for the Jetson TX2 and over 50,000 for the more recent Jetson AGX, thus exhaustively exploring all configurations at runtime, and even offline is impractical.
- *impactful:* given that, for each processing element (CPU, GPU, accelerators) and the memory controller the configurable frequency range has a range of 10× or more, the frequency choice has a significant impact on the application runtime and energy consumption (over 100× for TX2);
- *interdependent* - that is, the configuration choice for **each** individual component is essential to maximize application level objectives. To highlight this feature, Figure 3 demonstrates that the minimum possible energy consumption can only be achieved by configuring **all** CGM components. Taking Squeezenet as an example, the minimum energy consumption per inference is 101mJ. However, if we do not adjust **all** components, the energy consumption is higher. For instance, failing to configure the memory controller frequency, yet optimizing the configuration for both CPU and GPU results in a 46% increase in optimal energy consumption (146mJ per inference). Hence, to achieve optimal configuration, it is necessary to configure all components simultaneously.

3.2 The Impact of Thermal-Obliviousness

To highlight the shortcomings of *thermal-oblivious* frequency optimization frameworks, we emulate an optimal (yet *thermal-oblivious*) frequency optimization framework: the decisions of this framework are optimal in the sense that, at application startup, it chooses an optimal frequency configuration by evaluating *all* the possible configurations. However, this decision is static: the framework does not take into account the impact of temperature variations after the application's start-up.

We assume an energy-constrained scenario: the objective is to select the optimal CGM configuration (CGM_{optimal}) such that the energy budget per inference is not exceeded while maximizing performance (i.e., minimizing runtime per inference). The challenge is the variation in energy consumption and runtime at different platform temperatures. In other words, the energy and runtime of CGM_i at temperature θ_1 are different from those at temperature θ_2 ($energy^{\theta_1}_{CGM_i} \neq energy^{\theta_2}_{CGM_i}$ and $runtime^{\theta_1}_{CGM_i} \neq runtime^{\theta_2}_{CGM_i}$). Consequently, the optimal frequency configuration may depend on the platform's temperature ($CGM^{\theta_1}_{\text{optimal}} \neq CGM^{\theta_2}_{\text{optimal}}$). A *thermal-oblivious* framework, however, lacks the ability to adapt to thermal variations, and determines the optimal configuration at a fixed temperature at bootstrap without adjusting it later if the temperature changes.

We have implemented such a framework to study the effect of temperature variation on four different ML inference workloads. Figure 4 demonstrates two unfavorable scenarios:

- *Bootstraping at low temperature ($CGM^{25C}_{optimal}$) and using the optimization decision at a higher temperature:* This leads to *energy constraint violations* for all the applications (Figure 4). For instance, in the case of Squeezenet, if the framework selects the optimal configuration at 25°C, it satisfies the energy constraint (consumes less than 120mJ energy per inference). However, as the temperature increases, this CGM configuration's energy consumption becomes 140mJ, which is 20% higher than the energy budget.

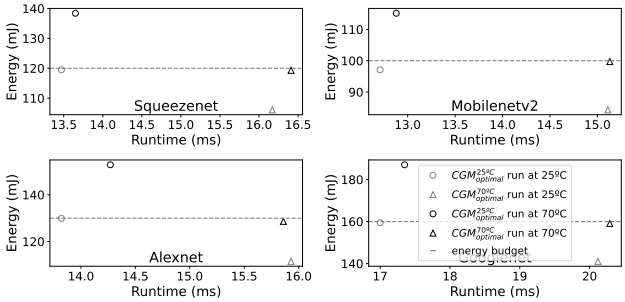


Figure 4: Temperature impact on *thermal oblivious* frequency optimization frameworks: (i) constraint violations, and (ii) sub-optimal solutions. The figure presents the effect of running a configuration that is optimal at one temperature (25 and 70°C) after the temperature changes (to 70, and respectively 25°C) and compares this with the actual optimal configuration. Circles represent running with the 25°C optimal CGM, while triangles represent running with the 70°C optimal CGM. Filled shapes indicate an ambient temperature of 25°C, and empty shapes represent 70°C. Four applications are presented under tight energy constraints (Table 4).

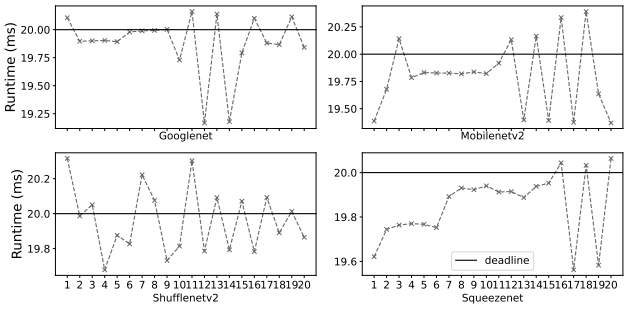


Figure 5: Plots present the application runtime of 20 consecutive inference tasks of different ML models at 70°C while using NeuOS with pre-populated 25°C characterization data as the frequency optimization framework. The QoS objective for all workloads is set to 50 FPS (i.e., 20 ms deadline).

- *Bootstraping at a high temperature ($CGM^{70C}_{optimal}$) and using the optimization decision at a lower temperature:* While this does not violate the energy constraints, it results in a *significant increase in runtime* compared to the optimal runtime. Again, using Squeezenet as an example, if the framework selects the optimal configuration at 70°C, the per-inference runtime is 16.2ms. However, when the temperature decreases, this configuration is 21% slower than the optimal configuration at 25°C (which has a runtime of 13.5ms).

3.3 Existing Work and Limitations of Thermal-Obliviousnes

Existing optimization frameworks can be classified based on two dimensions (Table 1):

- *Model-based vs. Model-free*: Model-based frameworks use explicit models (often obtained through a combination of platform characterization, prior knowledge, and analytical modeling) to perform the optimization task, while model-free frameworks optimize the frequency configuration based on the observations/feedback obtained by direct interactions with the environment without any prior knowledge.
- *Adaptive vs. Non-adaptive*. Adaptive frameworks attempt to adjust the frequency configuration at runtime, while non-adaptive frameworks find the optimal configuration only in an initial phase and do not change the configuration later.

Table 1: Optimization frameworks classification based on two dimensions: *(i) Model-based vs. Model-free, (ii) Non-adaptive vs. Adaptive. The asterisk (*) indicates that a framework is thermal-aware.*

Optimization Frameworks	Model-based	Model-free
Non-adaptive	MOSAIC[11]	Herti[14]
Adaptive	NeuOS[7], PredJoule[8], POET[10]	Autoscale[17], Alert[31], EdgeEngine*

Unfortunately, all current optimization frameworks (Table 1) have a common drawback - *thermal obliviousness*. Model-based non-adaptive frameworks are unable to perform well when facing platform temperature changes, as their models rely on information collected at a fixed temperature. Section §3.2 has demonstrated this for an optimal non-adaptive model-based framework. Figure 5 (and additional experiments in §6.1) highlight that even adaptive, frameworks perform sub-optimally when their design does not take temperature into account. Figure 5 focuses on an state-of-the-art adaptive, model-based (yet thermal-oblivious) framework: NeuOS [7], specifically designed to meet deadlines for autonomous vehicles using pre-characterized data. As the figure shows, NeuOS fails to effectively adapt to the platform's temperature changes. It exhibits multiple instances of deadline violations when tested with pre-populated data at 25°C and exposed to 70°C, highlighting its inability to handle temperature variations.

On the other hand, adaptive model-free frameworks attempt to adapt to constraint violations, including those caused by temperature changes without having any prior knowledge of the system. However, these frameworks suffer from two drawbacks due to the lack of platform temperature considerations in their design: *(i)* they do not retain any knowledge of previously observed temperatures. Consequently, each temperature change is treated as an entirely new environment, requiring the adaptation process to start from scratch. This limitation hinders their ability to achieve real-time adaptation and results in a high number of violations (demonstrated in Section 6.1). *(ii)* they may not adjust the frequency configuration if the constraint is not violated. As a result, the frameworks may settle for suboptimal configurations instead of actively seeking the best configuration (demonstrated in Section 6.1).

Table 2 summarizes the objectives thermal-oblivious frameworks fail to meet highlighting the advantages of EdgeEngine, our solution based on RL.

3.4 Thermal-Awareness through Reinforcement Learning (RL)

RL is a machine learning technique in which a computer agent learns to optimize a task via repeated interactions in a dynamic environment [21]. This learning approach enables the agent to

Table 2: Comparative Analysis of Edge Optimization Frameworks in Achieving each of the Thermal-Aware Framework Objectives

Framework	Approach	(O_1) Temperature Adaptation	(O_2) Cover Full Spage	(O_3) Real-Time Adaptation	(O_4) Runtime Noise Adaptation	(O_5) Lightweight Design	(O_6) Generaliz-ability
MOSAIC	*Non-adaptive Model-based*	×	×	×	×	×	×
NeuOS, PredJoule, POET	*Adaptive Model-based*	×	✔	×	✔	✔	×
Herti	*Non-adaptive Model-free (Deep RL)*	✔	✔	×	✔	×	✔
Autoscale	*Adaptive Model-free (Q-Learning)*	✔	×	×	✔	✔	✔
Alert	*Adaptive Model-free (Adaptive Kalman-filter)*	✔	✔	×	✔	✔	×
EdgeEngine	*Adaptive Model-free (Q-Learning)*	✔	✔	✔	✔	✔	✔

make a sequence of decisions that maximize a reward metric for the task without being programmed to optimize it.

Q-learning [32] is a widely used lightweight RL algorithm. It determines the best action based on the current state (s_t) of the system. It uses a Q-table where it stores the expected reward values for each action in any state. At each time step t, the agent randomly selects an action (a_t) for which it does not have any information (i.e., it performs *exploration*) or selects the action with the highest reward from the Q-table (i.e., it performs *exploitation*). The exploration/exploitation tradeoff is controlled by the parameter ϵ. Epsilon 'decay' can be applied, gradually reducing the exploration rate over time to prioritize exploitation as the agent gains more experience.

The Q-table is updated using the Bellman equation (Equation 1), with α as the learning rate and γ as the discount factor balancing immediate and future rewards. The reward value r_t reflects the outcome after performing an action in a given state, that is it reflects how good action a_t was for the state s_t.

$$Q^{new}(s_t, a_t) = (1-\alpha)Q(s_t, a_t) + \alpha(r_t + \gamma \underset{a'}{max} Q(s_{t+1}, a')) \quad (1)$$

While Q-learning has been previously applied in the context of frequency optimization frameworks for edge platforms [17], our innovation lies in making its application thermal aware. We achieve this by incorporating the platform temperature as an additional component in the state space, and introducing algorithmic heuristics to effectively address all objectives of the thermal-aware framework.

4 APPROACH: EDGEENGINE

We propose EdgeEngine (Figure 6), a *thermal-aware* optimization framework for edge inference. EdgeEngine uses a lightweight RL technique (Q-Learning [27]) (§3.4) to meet the application's objectives and adapt to temperature changes. Based on the application's constraints and the platform's temperature, EdgeEngine optimizes the frequency configuration of the CGM components.

EdgeEngine can adapt to two dimensions of application constraints: *deadline* and *energy* constraints. Users set the application's constraint dimension with the maximum permissible value. For an energy-constrained application, they set the maximum energy budget per inference, and for a deadline-constrained application, users set the target deadline for each inference. The RL agent selects a configuration that satisfies the constraint and optimizes on the unconstrained dimension.

We consider three main categories of objectives when designing EdgeEngine (mapped to O_{1-6} introduced in section 1).

1. High Accuracy Adaptation (O_{1-4})**.** EdgeEngine must choose the near-optimal frequency configurations with high accuracy at execution time. Selecting an incorrect configuration may cause: *(i)* constraint violations, which are unacceptable for real-time edge inference, and/or *(ii)* a sub-optimal configuration that results in higher energy consumption or lower performance.

EdgeEngine can adapt to new temperatures, without prior system knowledge (O_1). When EdgeEngine encounters a new temperature for the first time, it initiates an exploration phase (detailed in Section 4.2). In this phase, it explores the effect of various frequency configurations by adjusting all CGM frequencies to obtain near-optimal configuration (O_2). To effectively address the exploration-exploitation trade-off (Section 3.4), EdgeEngine employs the epsilon-greedy policy [30]. It gradually reduces the frequency of exploration actions and enters the exploitation phase by selecting the previously determined near-optimal configuration.

The exploration phase, however, can result in numerous constraint violations, given the large configuration space (Section 3.1), which is an intolerable outcome for several edge applications. To address this issue, we propose two domain-specific heuristics (detailed in Section 4.2) based on our temperature characterization: *(i)* action pruning and *(ii)* escape action. These enhancements give EdgeEngine real-time adaptation capabilities (O_3) by expediting the exploration process, and minimizing the number of violations incurred during exploration.

Even during the exploitation phase after EdgeEngine has selected what appeared as a near-optimal configuration, variability in power and runtime can lead to constraint violations. To mitigate this issue (O_4), EdgeEngine continuously monitors the inference metrics: if a constraint violation is detected, EdgeEngine promptly adjusts the frequency configuration to increase the guardband which protects it against variability in runtime or power.

2. Lightweight Design (O_5)**.** EdgeEngine should impose minimal latency overhead to avoid increasing the ML inference time. It should also have minimal computation and memory overheads to be compatible with resource-constrained edge platforms. To minimize latency overhead, we employ Q-Learning instead of alternative RL techniques such as Deep RL (DRL) [21] or Temporal Difference Learning (TD Learning) [27]. Q-Learning's utilization of a lookup table called the *Q-Table* allows for efficient action selection with minimal latency. This lightweight algorithm also requires limited computational resources [17], making it well-suited for our objective. Furthermore, we ensure EdgeEngine has minimal memory overhead, by storing the Q-Table in compact format.

3. Generalizability (O_6)**.** EdgeEngine aims to be versatile and compatible with diverse environments, applications, and constraints. The model-free nature of Q-Learning [26], enables EdgeEngine to adapt seamlessly to different environments and applications. Unlike methods that rely on pre-existing knowledge about system dynamics [7, 8, 10, 11], Q-Learning learns directly from interactions with the environment, enabling EdgeEngine to dynamically adjust its behavior and configuration based on its specific operational context.

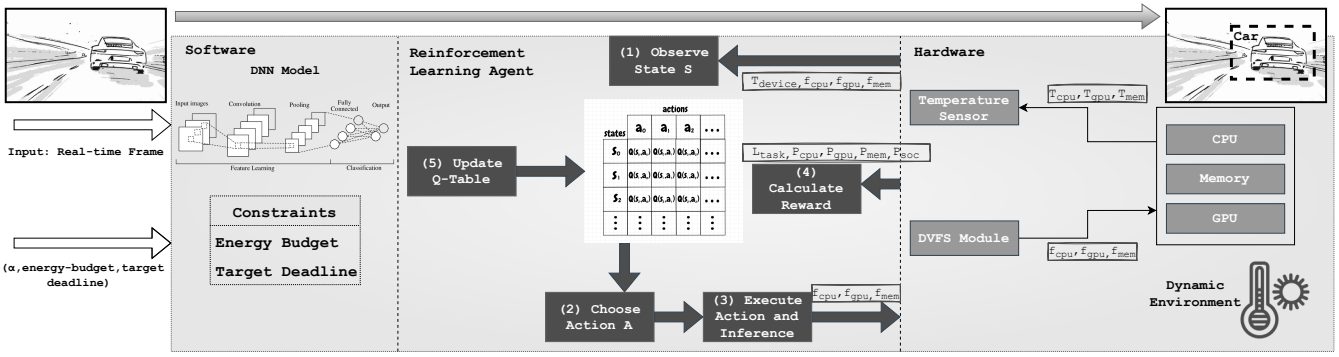


Figure 6: EdgeEngine overview. EdgeEngine flow can be summarized into five steps: *(i)* Determines the current state (s_t) by obtaining device temperature and frequency configuration, *(ii)* In *exploitation* mode it selects the action with the highest expected reward for s_t from Q-table, while in *exploration* mode it picks a random action; *(iii)* Executes the action by adjusting the CGM frequency levels, and performs inference, *(iv)* Estimates power and runtime for the inference, and calculates the reward value, and *(v)* Updates Q-table with reward value

4.1 EdgeEngine Design

This section presents the main components of EdgeEngine's RL agent: the *State*, *Action*, and *Reward* introduced in §3.4.

State. EdgeEngine state s_t at time t is captured by the following tuple: $\{f_C^t, f_G^t, f_M^t, T^t\}$. f_C^t, f_G^t, and f_M^t are the CPU, GPU, and memory controller frequency levels. Capturing the current frequency configuration helps the agent choose the best action by deciding on the next frequency configuration.

T^t is the current board's temperature (in our experiments, the average temperature of the CPU (T_C^t) and GPU (T_G^t) components, obtained from the onboard thermal sensor). To make the temperature, a continuous metric, compatible with the Q-learning process based on the Q-table, we quantize it to discrete ranges (detailed in §5).

Action. EdgeEngine's possible actions at each state s_t are a single frequency level increase or decrease in one of the CPU, GPU, or memory controller components, or to do nothing (DN), which indicates no change in the frequency levels.

$$\{INC_C, INC_G, INC_M, DEC_C, DEC_G, DEC_M, DN\}$$

To thoroughly explore the frequency state space and find near-optimal configuration, EdgeEngine *(i)* only adjusts the frequency level of only one component as a single action, and *(ii)* employs the ϵ-greedy approach to choose the between exploration and exploitation.

Reward. Equation 3 presents the reward function that the RL agent employs.

$$\begin{cases} \phi & in\ case\ of\ constraint\ violation \\ \Omega.(\frac{1}{runtime}) + (1-\Omega).(\frac{1}{energy}) & otherwise \end{cases} \quad (2)$$

In Equation 3, $\Omega \in \{0, 1\}$ indicates whether the application is energy critical or deadline critical. Users set this value in EdgeEngine's configuration based on the application requirement. $\Omega = 0$ represents the application is deadline-constrained, while $\Omega = 1$ means it is energy-constrained. The first goal of the reward function is to avoid violating the application's constraint. Regarding the constraint's dimension, users set the target deadline or energy budget per inference. Suppose the agent observes a constraint violation. In that case, it sets the reward value as a negative pre-configured value ϕ, Otherwise, the reward is positive and indicates how good the action is for the dimension without a critical constraint, e.g., the reward is higher if the action is more energy-efficient for a deadline-constrained application.

Algorithm 1 EdgeEngine Algorithm

1: $Q \leftarrow$ initializeQTable()
2: $\alpha, \gamma, \epsilon \leftarrow initializeVariables()$
3: **repeat**
4: Observe state s_t
5: $A_t \leftarrow$ getAvailableActions(s_t)
6: **if** generateRandNumber() $< \epsilon$ **then**
7: $a_t \leftarrow$ getRandomAction(A_t)
8: **else**
9: $a_t \leftarrow argmax_{a \in A_t} Q(s_t, a)$
10: runInferenceByAction(a_t)
11: $E_t, L_t \leftarrow$ measureLatencyEnergy()
12: $R_t \leftarrow$ calculateReward(E_t, L_t)
13: Observe state s_{t+1}
14: $a' \leftarrow argmax_{a \in validActions} Q(s_{t+1}, a)$
15: $Q(s_t, a_t) \leftarrow (1-\alpha)Q(s_t, a_t) + \alpha(R_t + \gamma Q(s_{t+1}, a'))$
16: $escapeAction(S_t, a_t, R_t)$
17: $\epsilon^T \leftarrow$ updateEpsilon(ϵ^T)
18: $s_t \leftarrow s_{t+1}$

Algorithm 1 shows the RL algorithm used by EdgeEngine.

4.2 EdgeEngine Algorithm

Exploration-Exploitation Trade-off. To adapt to different platform temperatures and control the exploration-exploitation trade-off, EdgeEngine employs the epsilon-decay approach. It maintains a variable ϵ^T, for each platform temperature state (T), the higher this value is the greater the likelihood that the chosen action is an exploration one. At the end of each iteration (line 17), the value of ϵ^T is decreased. To determine the action selection, EdgeEngine

generates a random number and compares it with the current value of ϵ^T. If the random number is greater, the algorithm *'explores'*: that is, it chooses a new random action. Otherwise, it *exploits*: it uses the best valid action from the Q table (line 6-9).

Domain-specific Heuristic Enhancements. To address the challenges posed by the large exploration space and the occurrence of violations during the exploration phase, we have developed two novel domain-specific techniques to accelerate the adaptation process and minimize violations: *(i)* action pruning and *(ii)* escape action.

Action Pruning: At each state, EdgeEngine prunes the actions that lead to violations (i.e., prohibited actions), and selects actions from a set of available actions (line 5). EdgeEngine keeps a list of prohibited actions for each CGM frequency configuration, and marks an action as prohibited if:

$$a^T_{CGM}\ is\ prohibited \mid \exists \kappa \leq T \Rightarrow a^\kappa_{CGM}\ violates\ QoS \quad (3)$$

An action is prohibited in a specific CGM if it previously caused violations at the current temperature or at lower temperatures. This prohibition mechanism is based on the characterization study (§2) that shows that increasing the temperature leads to higher runtime and energy consumption. This action pruning leads to decreasing the search space size, as EdgeEngine observes more temperatures, which leads to faster adaptation, and fewer violations during exploration (we evaluate these improvements in Section 6.2).

Escape Action: In the event of a constraint violation, EdgeEngine promptly adjusts the frequency configuration to bring it closer to the optimal configuration (line 16), reducing the likelihood of future violations. For applications with deadline constraints, EdgeEngine increases the frequency configuration by two levels rather than one, if a deadline is missed. This approach is based on our observation that increasing the frequency generally leads to performance improvement.

However, our characterization study reveals that the impact on energy consumption is not as straightforward. In some cases, increasing the frequency leads to higher energy consumption, while in others, it has the opposite effect. This is because energy consumption depends on the product of power and runtime, and the influence of frequency changes on these factors is in opposite directions. Therefore, we do not perform this optimization for energy-constrained applications.

5 EXPERIMENTAL METHODOLOGY AND SETUP

This section presents our experimental methodology (§5.4), the rationale behind the evaluation scenarios we explore (§5.5), as well as the experimental platform (§5.1), Edge engine reinforcement-learning specifies parametrization (§5.2) and workloads (§5.3).

5.1 Experimental Platform

Hardware Platform. We employ the NVIDIA Jetson TX2, a heterogeneous embedded edge platform specifically designed to enhance the performance of machine learning applications at the edge. The Jetson TX2 integrates the power of the NVIDIA Pascal GPU with low-power ARM CPU cores, using a shared-memory architecture.

Table 3: Jetson TX2's hardware specifications.

Jetson TX2	
CPU	Dual-Core NVIDIA Denver 2 64-Bit CPU Quad-Core ARM® Cortex®-A57 MPCore
GPU	256-core NVIDIA Pascal™ GPU architecture with 256 NVIDIA CUDA cores
Memory Controller	8GB 128-bit LPDDR4 Memory 1866 MHx - 59.7 GB/s
Storage	32GB eMMC 5.1

This combination of GPU and CPU cores enables energy-efficient execution of low-power ML tasks, making it an ideal platform for our purposes. (Table 3).

Software Stack. We use NVIDIA Jetpack v5.0.2, which has the latest Jetson Linux (v34.1.1) and CUDA v11.4. We use PyTorch v1.6 as our deep learning framework.

5.2 RL Parameters Specific to EdgeEngine

Hyperparameters. To determine the optimal values for the hyperparameters learning rate (α) and discount factor (γ), we conduct a sensitivity analysis. We consider three classes of values for each parameter: low (0.2), medium (0.5), and high (0.8). EdgeEngine is being trained on Mobilenetv2 using all nine combinations of these values, resulting in a total of nine experiments. Through our analysis, we observe that the agent always performs most effectively when the learning rate is set to a high value and the discount factor to a low value.

Exploration Rate. We set the maximum ϵ value as 0.995, and the minimum value as 0.01. We also use the epsilon decay technique by multiplying by the maximum ϵ value at the end of each round. This approach allows for a gradual reduction of exploration and an increased focus on exploitation as the experiment progresses.

State Quantization. EdgeEngine state is determined by the current CGM Frequency level and temperature. We use the Nvidia Jetson TX2 frequency levels: 12, 11, and 13 frequency levels for CPU, GPU, and memory controller respectively [28]. We discreteize the operating temperature range of Jetson TX2 [29] (-25°C to 80°C) into 21 levels, each corresponding to a 5°C temperature range.

5.3 Workloads

We focus on Convolutional Neural Network (CNN) inference. CNNs are chosen as the benchmark since: *(i)* they are widely utilized in computer vision tasks, including object detection and object classification, which are commonly encountered in edge ML vision applications [22], and *(ii)* many popular ML benchmarks, such as MLPerf [25] and EdgeBench [9], prominently feature CNN inference as their primary building blocks. To provide a comprehensive analysis, we select seven pre-trained CNN models: Mobilenetv2, Alexnet, Resnet, Inception3, Shufflenetv2, Googlenet, and Squeezenet, which are representative of a wide range of CNN architectures commonly used in ML applications.

5.4 Methodology: Emulating Temp.-Controlled Experiments

Our evaluation of EdgeEngine and other frameworks must consider various applications, constraints, and ambient temperature change

Table 4: The loose and tight energy budgets and the target deadlines for different ML inference applications for evaluating EdgeEngine.

Application	Loose Constraint		Tight Constraint	
	Energy Budget (mJ)	Target Deadline (ms)	Energy Budget (mJ)	Target Deadline (ms)
Alexnet	110	18	100	13
Googlenet	200	35	160	18
Inception3	590	76	505	60
Mobilenetv2	110	19	100	14
Resnet	170	20	150	15
Squeezenet	140	21	120	13
Shufflenetv2	100	24	80	16

scenarios (see Section 6). However, conducting experiments in a temperature chamber is time-consuming, as we need to wait, approximately 5 to 10 minutes, for the chamber to reach the desired temperature before each test (and, additionally, we cannot remotely control the chamber). To mitigate this issue, we have developed a software emulator to simulate the effect of temperature changes on application runtime and power consumption.

The emulator is seeded with the runtime and power consumption distributions for each CGM frequency configuration at 5°C intervals between 25°C and 70°C. This data is obtained by conducting experiments in the temperature-controlled chamber (and required approximately two days of continuous runtime for each model). Although this initialization process incurs an upfront cost, it eliminates the need for repeated experiments in a temperature-controlled chamber.

To enhance the realism of the emulator, we model the runtime variability we observed empirically. We characterize this variability using 252 empirically collected distributions, and observe that the majority follow a General Extreme Distribution (GEV), so we incorporate GEV runtime noise into the emulator to reflect the intrinsic runtime variability.

If the emulator lacks data for a specific temperature, it approximates the missing values using linear interpolation based on available adjacent data points.

5.5 Methodology: Evaluation Scenarios

We evaluate EdgeEngine and compare it to related work (in Section 6) in the context of two evaluation scenarios: *loose* and *tight* constraints (for deadlines or energy). To determine the actual values of the constraints (listed in Table 4), we consider the energy/runtime behavior of an application when running on the Jetson TX2 at 65°C[4].

- The *tight* constraint is defined as 10% higher than the lowest achievable energy/runtime, ensuring a stringent constraint that leads to a deep space exploration for satisfiability.
- The *loose* constraint, on the other hand, is set as 10% lower than the median energy/runtime of running the application and, since a large set of frequency configurations satisfies this constraint, this scenario verifies the ability of an optimization framework to optimize in the unconstrained direction.

[4]By choosing 65°C, we guarantee the constraints satisfaction achievability in all scenarios, as we raise the ambient temperature up to 65°C in our experiments.

6 EVALUATION

This section presents the experimental evaluation of EdgeEngine in the context of different constraint scenarios (§5.5) and applications (§5.3). First, we focus on EdgeEngine's ability to satisfy the applications' constraints and meet the optimization objective as temperature changes (§6.1). We evaluate EdgeEngine's efficiency by comparing it to an idealized oracle. We highlight the importance of *thermal-awareness* by comparing EdgeEngine with state-of-the-art frequency optimization frameworks, all *thermal-oblivious* as well as with thermal-oblivious versions of EdgeEngine. Finally, we assess EdgeEngine's practicality (§6.2) by examining how it satisfies thermal-aware framework objectives (O_{1-6}).

6.1 Adaptability and Efficiency

We assess EdgeEngine's adaptability to different temperatures by evaluating two metrics: the constraint violation rate, and the quality of the solution in the unconstrained dimension.

To conduct the experiment, we use the following temperature variation scenario: Initially, we set the ambient temperature to 25°C and incrementally increase it to 65°C, subsequently decreasing it back to 25°C. At each interval of 10°C, we allow for a stable temperature period of 500 inferences before changing the temperature again. During the experiment, we record the number of violations, and, at the end of the experiment, we estimate the runtime and energy consumption of the final configuration for each temperature level.

Our chosen methodology aims to thoroughly test the adaptability of EdgeEngine across different temperature ranges, both when encountering them for the first time (on temperature increase) and when already observed them (on temperature decrease). This methodology also evaluates EdgeEngine in a challenging environment where temperatures fluctuate rapidly, with each 10°C temperature increase or decrease in 5 seconds. However, we also conducted tests in more relaxed environments where temperature changes occur gradually. In these scenarios, we observed that EdgeEngine exhibited improved performance as the temperature evolved more slowly. This favorable outcome can be attributed to EdgeEngine's thermal awareness and embedded heuristics, which enable it to adapt and optimize its operations accordingly.

We run EdgeEngine on both deadline and energy-constrained evaluation scenarios (§5.5), and compare to both an idealized *ORACLE* and to related work :

- *ORACLE:* We implement a perfect knowledge ORACLE that finds the optimal CGM configuration based on runtime and energy measurements pre-recorded at the bootstrap time for each temperature level. Note that the ORACLE is not affected by power and runtime variability (it is an idealized solution): it has full knowledge of the future power and runtime before choosing a configuration. We compare EdgeEngine with the ORACLE to assess how efficiently EdgeEngine optimizes runtime and energy, as well as how closely its configurations align with the optimal solution.
- *NeuOS:* an adaptive model-based framework designed for deadline-constrained scenarios only. NeuOS is based on an extensive characterization at deployment time, yet it is thermal-oblivious: this characterization is done at fixed temperature.

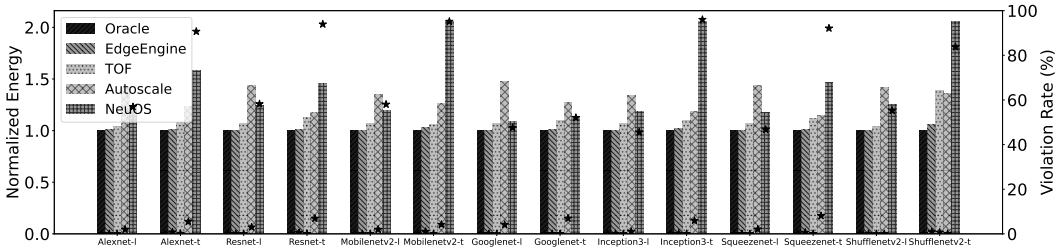


Figure 7: Runtime-constrained Scenario Comparison. The plot shows the energy consumption for various optimization frameworks and seven different workloads with both loose and tight deadline constraints. Energy is normalized to that of the Oracle (left y-axis, lower is better, 1 is ideal). The plot also presents the quality of each framework: that is, the rate of the runtime constraint violations (right y-axis, lower is better). The reported results are the average of ten separate runs for each experiment. Lower values are better.

We implement the NeuOS framework based on the description by Bateni et al [7] and seed it with data at 25C. By comparing to NeuOS, we can find how good online-adaptive solutions, specifically EdgeEngine, are in terms of constraint (i.e., deadline) satisfaction.

- *Autoscale:* an adaptive model-free optimization framework based on RL designed for deadline-constrained scenarios. NeuOS has two shortcomings: *(i)* is thermal-oblivious (it does not consider the platform temperature) and, additionally, *(ii)* it does not aim to adjust the memory controller frequency (it only controls the CPU and GPU frequencies). We implement the Autoscale agent following the description by Kim et al [17]. As a validation, we ensure that it can find the best configuration (as per its design) when the temperature remains constant. By comparing to Autoscale, we *(i)* highlight that an adaptive model-free optimization framework must be thermal-aware to be effective; and *(ii)* study the significance of adjusting *all* CGM configurations.
- *TOF:* a thermal-oblivious version of EdgeEngine. We modify EdgeEngine's RL agent to remove temperature as an input, rendering it thermal-oblivious. By comparing EdgeEngine to TOF, we: *(i)* can pinpoint the specific limitations of thermal-oblivious frameworks (e.g., AutoScale) and why they cannot effectively operate, even when adjusting all CGM configurations; and *(ii)* we have a thermal-oblivious comparison base for energy-constrained scenarios as none of the previous work was designed with this scenario in mind.

Evaluation results for the deadline-constrained scenario. Figure 7 presents the energy consumption and violation rates of different frameworks in deadline-constrained scenarios with tight and loose deadlines. Experiments reveal that:

- *EdgeEngine* works well in terms of constraint satisfaction and energy efficiency. It exhibits a mere 3% higher energy consumption on average (with a maximum of 5%) compared to the ORACLE. It also has a maximum violation rate of only 1% across all experiments.
- *Autoscale* and *TOF* exhibit relatively low violation rates (2% and 5% respectively) as they are adaptive model-free frameworks designed to mitigate deadline violations. However, their energy consumption efficiency is compromised, with TOF averaging 10% higher energy consumption (up to 38% in some cases) and Autoscale averaging 32% higher consumption (up to 47%) compared to EdgeEngine. The higher energy consumption of Autoscale compared to TOF is primarily due to its inability to adjust memory frequency. Autoscale and TOF energy inefficiency shows that RL-based thermal-oblivious frameworks cannot adapt well to environments with varying temperatures.
- *NeuOS* exhibits the poorest energy efficiency and highest violation rate due to its utilization of bootstrap data at a fixed temperature within a model-based framework, which is inadequate in varying temperature environments.

Evaluation results for the energy-constrained scenario. Figure 8 shows the runtime efficiency and violation rates of energy-constrained applications. Experiments reveal that:

- *EdgeEngine* has a maximum violation rate of 1% throughout the experiments, with a runtime only 2% higher than that of the ORACLE. This shows that EdgeEngine can adapt well to varying temperatures and finds near-optimal solutions.
- *TOF*, conversely, exhibits an average violation rate of 41%. This notable contrast arises from the substantial energy variations associated with temperature and the vast frequency space, making it challenging for an adaptive framework to escape violations if it lacks thermal awareness.

Summary. Figure 9 summarizes our experimental results so far, and highlights EdgeEngine's adaptability to fluctuating ambient temperatures with a minimal number of violations, effectively selecting configurations that closely align with optimal solutions. Notably, EdgeEngine exhibits remarkable proximity to the ORACLE in both deadline and energy-constrained applications. In contrast, other thermal-oblivious frameworks are (1) inefficient (TOF and Autoscale in deadline-constrained applications) or (2) show a high violation rate (TOF in energy-constrained applications), or (3) suffer from both inefficiency and a high violation rate (NeuOS).

6.2 Practicality

This section dives deeper in the previous experiment to analyze EdgeEngine's practicality in real environments by studying how it satisfies the objectives (O_{1-6}) thermal-aware frameworks. To this end, we analyze the previous experiment 's two phases separately and we discuss EdgeEngine overheads:

Temperature-increase phase. *Temperature-increase*: In this phase, we gradually increase the temperature from 25 to 65°C. EdgeEngine encounters each temperature for the first time and adopts a high exploration rate to search for near-optimal configurations.

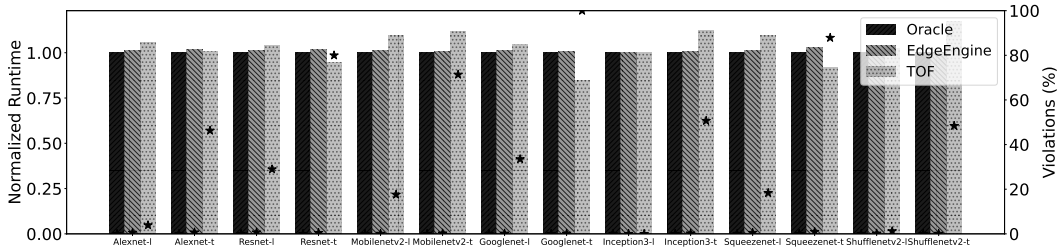


Figure 8: Energy-constrained Scenario Comparison. The plot shows the normalized runtime (left x-axis) of various optimization frameworks compared to the Oracle for seven different workloads (inference models), while adhering to both loose and tight energy constraints. Lower normalized runtime is better (1 is the optimal). The plot also displays the rate of violations of the energy constraint for each framework (lower is better, note that some frameworks that appear to perform well on the optimization criteria have unacceptably high rates of constraint violations). The reported results are the average of ten separate runs for each experiment. Lower values are better.

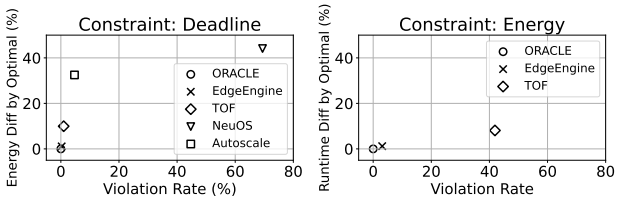


Figure 9: Comparison summary. The figures present the average violation rate (x-axis) and the quality of the optimization choice (y-axis) for runtime-constrained (left plot) and energy-constrained scenarios (right plot). Values are averages over all experiments presented in Figures 7 and 8. Values closer to the point (0,0) are better.

Table 5: The average number of violations during training for the optimized and vanilla versions of EdgeEngine across all seven models at various temperatures when board temperature increase from 25 to 65°C.

EdgeEngine Version	Constraint: Deadline				
	25°C	35°C	45°C	55°C	65°C
EE-Vanilla	83.14	91.43	75.14	74.14	87.86
EE-Optimized	21.86	4.86	4.14	2.71	2.14
Change (%)	74%	95%	94%	96%	98%
EdgeEngine Version	**Constraint: Energy**				
	25°C	35°C	45°C	55°C	65°C
EE-Vanilla	51.14	64.00	78.14	111.00	145.29
EE-Optimized	15.43	21.71	37.29	54.71	48.43
Change (%)	70%	66%	52%	51%	67%

EdgeEngine must demonstrate real-time adaptation capabilities (O_3) to avoid inadequate adaptation (i.e., far from optimal, multiple violations) when triggered by constraint violations as the platform first observes a temperature level. We focus on evaluating the effectiveness of the heuristics we have incorporated into the EdgeEngine RL agent (§4.2), namely *action pruning* and *escape action*. To this end, we measure the number of violations at each temperature and compare it with a version of EdgeEngine without these heuristics, *EE-Vanilla*.

The number of violations for both EdgeEngine versions (Vanilla and Optimized) is presented in Table 5. For deadline-constrained applications, during the initial experiment at 25°C without any prior knowledge, EE-Optimized observed an average of 22 violations during the adaptation process, which is a significant reduction of 74% compared to EE-Vanilla. As the temperature increased, EE-Optimized consistently exhibited significantly fewer violations at each temperature level than EE-Vanilla. This is due to two factors: *(i)* EE-Optimized effectively transfers knowledge to higher temperatures through action pruning, and *(ii)* the insignificant runtime variations with temperatures (lower than 5%) make optimal CGM configurations close to each other. For energy-constrained applications, the optimizations in EE-Optimized resulted in a halving of the number of violations. However, the number of violations is higher than the deadline-constrained scenario. This is due to the substantial variation in energy consumption across different temperatures, which requires more exploration by EdgeEngine to find near-optimal solutions.

Temperature-decrease phase. In this phase, EdgeEngine has already explored the current temperature and acquired knowledge of the near-optimal configuration. Thus, during this phase, EdgeEngine primarily exploits the acquired knowledge. As the platform temperature varies, EdgeEngine needs to promptly adjust its configuration (O_1). Additionally, the presence of runtime noise can potentially lead to violations (O_4). We analyze how EdgeEngine adapts to temperature variations and handles the challenge posed by runtime variability.

In this phase, EdgeEngine generates only a few constraint violations, averaging 2 (with a maximum of 9) violations, across all experiments. We attribute these violations to intrinsic runtime noise. Figure 10 illustrates EdgeEngine's dynamic adaptation in an energy-constrained scenario. We showcase three representative applications: a lightweight (Mobilenetv2), a medium-weight (Googlenet), and a heavyweight model (Inception3). Figure 10 demonstrates that EdgeEngine swiftly adjusts the CGM frequencies to efficiently restore constraint satisfaction in the presence of runtime noise. For instance, when considering Mobilenetv2 at 65°C, around the 250th frame, EdgeEngine begins to experience runtime variability, leading to increased energy consumption, and also a violation. However, EdgeEngine promptly adapts its configuration, and no further violations occur at this temperature. Additionally, as temperature decreases, EdgeEngine adjusts the configuration and selects near-optimal settings. This adaptation leads to a reduction in runtime,

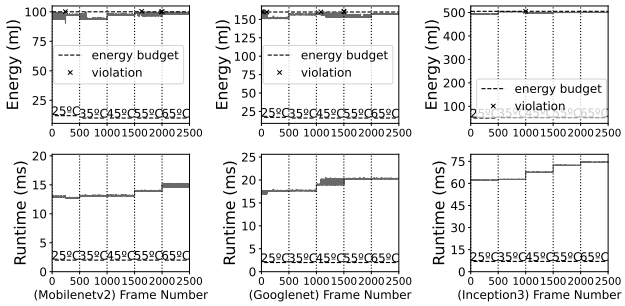


Figure 10: The figures illustrate adaptation to previously observed platform temperatures ranging from 65 to 25ºC and runtime variability, while operating under tight energy budget constraints for three CNN applications. The upper plots depict the energy consumption (including any violations as crosses), while the lower plots showcase the corresponding runtime for each inference. Lower values are better.

as lower temperatures correspond to lower energy consumption, thereby enhancing performance.

Overheads. We estimate EdgeEngine runtime and memory overhead to show it is practical for deployment in resource-constrained environments (O_5). Note that our Python-based implementation of EdgeEngine is not optimized - thus our estimates are an upper bound of overheads in a realistic deployment. The runtime overhead of EdgeEngine on Jetson TX2, on average across all the models, is 340μs. This overhead is the total execution time of selecting an action, calculating the reward, and updating the Q-table. Compared to the Squeezenet execution time (13ms), the fastest model in our experiment, this overhead constitutes 2.61% of the total inference latency. Regarding memory overheard, the memory usage of EdgeEngine is 16MB, on average, over the applications. This is significantly lower than the memory footprint of the applications, which often use GBs of data.

To conclude, EdgeEngine can swiftly adapt to temperature changes (O_1) and runtime noise (O_3) by adjusting all components (O_2). It also exhibits real-time adaptation capabilities by having a low number of violations when it observes a temperature for the first time (O_4). EdgeEngine has the above ability across temperatures, applications, and constraints (O_5) and maintains low runtime and memory overheads (O_6).

7 DISCUSSION

Extending the problem space. This study focuses on highlighting the benefits of *thermal-awareness* and proposes RL as a possible technique to support it. To this end, we focus on the problem frequency optimization frameworks face: choosing an optimal frequency for each processing element such that one constraint in the 2D space (*time energy*) is satisfied while the other one is optimized. Edge applications, however, may operate in a space that has additional dimensions (e.g., *accuracy* for inference applications [15]) and may have additional controls (e.g., the number of cores used to control the time/energy trade-off, or application-specific configurations such as weight quantization/pruning [24] to control accuracy in CNNs). We believe that *thermal-awareness* continues to be a salient issue even in a multidimensional constraint space with multiple controls, and that RL will continue to be an effective solution.

Are the energy savings obtained meaningful? One might ask how important it is to optimize energy for edge platforms compared to the total energy consumption of the devices they are deployed on. For one perspective, we can compare edge platform power consumption to the power consumption of small-sized drones [1]. These operate within a comparable power range (i.e., around 20 watts) as current edge platforms when running at maximum frequency [28]. Thus, optimizing the energy consumption of the edge platform becomes an important element to ensure sustainable operation.

Controlling the onboard fan speed. A notable aspect of current edge platforms is the presence of the onboard fan, which is controlled by the platform's operating system. Based on our observations, on the Jetson TX2, the fan operates at the maximum speed above 50ºC. To eliminate this source of variability, our characterization and experiments control the fan to be on at all times. An important avenue for future work is the extension of EdgeEngine capabilities to include control over the onboard fan speed. This would require the framework to learn and understand the impact of different fan speeds at various states (such as workload, temperature, and configuration) on power consumption and runtime.

Why not using deep reinforcement learning instead of Q-Learning? The key challenge with employing modern *deep* reinforcement learning models [18] for adapting to constraints is their substantial computational demands, rendering them impractical for resource-constrained edge platforms. For instance, both Herti [14] and zTT [16] rely on deep reinforcement learning but follow a training approach that occurs on a separate server, with the trained model subsequently transferred to the edge platform. This method lacks real-time adaptability, as it can take 30-183 milliseconds to determine the optimal action for zTT (significantly more compared to EdgeEngine). Furthermore, the adaptation process, involving the training of the deep Q-Network, is relatively slow, surpassing 300s for zTT, while EdgeEngine completes this process in just 15s.

8 SUMMARY

We demonstrate the effects of temperature variation on current *thermal-oblivious* frequency optimization frameworks showing that they either violate the application's constraints or select performance/energy-inefficient configurations. To address this challenge, we present *EdgeEngine*, a lightweight optimization framework that adapts to the environment's temperature and the application's constraints using RL. Compared to thermal-oblivious frameworks, EdgeEngine *(i)* selects frequency configurations that satisfy the applications' constraints, *(ii)* reduces violations by an average of *41%* with an *8%* lower runtime for energy-constrained applications, and *(iii)* reduces violations by *26%*, with *29%* lower energy consumption for deadline-constrained applications.

Acknowledgements: This work was supported in part by grants from Huawei, and the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grants Program.

REFERENCES

[1] 2023. Mavic Mini Drone - Specs. https://www.dji.com/ca/mavic-mini/downloads

[2] 2023. Temperature chamber - HP-100V-T. https://www.mrclab.com/Media/Doc/HP-FP-LP-TP-V-Tseries_SPEC2022.pdf

[3] Hazem A. Abdelhafez, Hassan Halawa, Mohamed Osama Ahmed, Karthik Pattabiraman, and Matei Ripeanu. 2021. MIRAGE: Machine Learning-based Modeling of Identical Replicas of the Jetson AGX Embedded Platform. In *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. 26–40. https://doi.org/10.1145/3453142.3491284

[4] Hazem A. Abdelhafez, Hassan Halawa, Amr Almoallim, Amirhossein Ahmadi, Karthik Pattabiraman, and Matei Ripeanu. 2022. Characterizing Variability in Heterogeneous Edge Systems: A Methodology Case Study. In *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*. 107–121. https://doi.org/10.1109/SEC54971.2022.00016

[5] Hazem A. Abdelhafez, Hassan Halawa, Karthik Pattabiraman, and Matei Ripeanu. 2021. Snowflakes at the Edge: A Study of Variability among NVIDIA Jetson AGX Xavier Boards. In *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking* (Online, United Kingdom) *(EdgeSys '21)*. Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/3434770.3459729

[6] Amirhossein Ahmadi, Hazem Abdelhafez, Shashwat Jaiswal, Karthik Pattabiraman, and Matei Ripeanu. 2023. Hot Under the Hood: An Analysis of Ambient Temperature Impact on Heterogeneous Edge Platforms. In *Proceedings of the 6th International Workshop on Edge Systems, Analytics and Networking* (Rome, Italy) *(EdgeSys '23)*. Association for Computing Machinery, New York, NY, USA, 25–30. https://doi.org/10.1145/3578354.3592868

[7] Soroush Bateni and Cong Liu. 2020. NeuOS: A Latency-Predictable Multi-Dimensional Optimization Framework for DNN-driven Autonomous Systems. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 371–385. https://www.usenix.org/conference/atc20/presentation/bateni

[8] Soroush Bateni, Husheng Zhou, Yuankun Zhu, and Cong Liu. 2018. PredJoule: A Timing-Predictable Energy Optimization Framework for Deep Neural Networks. In *2018 IEEE Real-Time Systems Symposium (RTSS)*. 107–118. https://doi.org/10.1109/RTSS.2018.00020

[9] Anirban Das, Stacy Patterson, and Mike Wittie. 2018. EdgeBench: Benchmarking Edge Computing Platforms. In *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*. 175–180. https://doi.org/10.1109/UCC-Companion.2018.00053

[10] C. Imes et al. 2015. POET: a portable approach to minimizing energy under soft real-time constraints. In *IEEE RTAS*. 75–86.

[11] M. Han et al. 2019. MOSAIC: Heterogeneity-, Communication-, and Constraint-Aware Model Slicing and Execution for Accurate and Efficient Inference. In *IEEE PACT*. 165–177.

[12] N. Razali et al. 2011. Power comparisons of shapiro-wilk, kolmogorov-smirnov, lilliefors and anderson-darling tests. *Journal of statistical modeling and analytics*, 21–33.

[13] P. Chestovich et al. 2022. Temperature Profiles Of Sunlight-Exposed Surfaces In A Desert Climate: Determining The Risk For Pavement Burns. *Journal of Burn Care & Research: Official Publication of the American Burn Association*, irac136–irac136.

[14] Myeonggyun Han and Woongki Baek. 2021. HERTI: A Reinforcement Learning-Augmented System for Efficient Real-Time Inference on Heterogeneous Embedded Systems. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 90–102. https://doi.org/10.1109/PACT52795.2021.00014

[15] Hyuk-Jin Jeong, Hyeon-Jae Lee, Chang Hyun Shin, and Soo-Mook Moon. 2018. IONN: Incremental Offloading of Neural Network Computations from Mobile Devices to Edge Servers. In *Proceedings of the ACM Symposium on Cloud Computing* (Carlsbad, CA, USA) *(SoCC '18)*. Association for Computing Machinery, New York, NY, USA, 401–411. https://doi.org/10.1145/3267809.3267828

[16] Seyeon Kim, Kyungmin Bin, Sangtae Ha, Kyunghan Lee, and Song Chong. 2022. ZTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices. *GetMobile: Mobile Comp. and Comm.* 25, 4 (mar 2022), 30–34. https://doi.org/10.1145/3529706.3529714

[17] Young Geun Kim and Carole-Jean Wu. 2020. AutoScale: Energy Efficiency Optimization for Stochastic Edge Inference Using Reinforcement Learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1082–1096. https://doi.org/10.1109/MICRO50266.2020.00090

[18] Yuxi Li. 2017. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274* (2017).

[19] Bingqian Lu, Jianyi Yang, Lydia Y. Chen, and Shaolei Ren. 2019. Automating Deep Neural Network Model Selection for Edge Inference. In *2019 IEEE First International Conference on Cognitive Machine Intelligence (CogMI)*. 184–193. https://doi.org/10.1109/CogMI48466.2019.00035

[20] Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. 2018. Taming Performance Variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 409–425. https://www.usenix.org/conference/osdi18/presentation/maricq

[21] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).

[22] M. G. Sarwar Murshed, Christopher Murphy, Daqing Hou, Nazar Khan, Ganesh Ananthanarayanan, and Faraz Hussain. 2021. Machine Learning at the Network Edge: A Survey. *ACM Comput. Surv.* 54, 8, Article 170 (oct 2021), 37 pages. https://doi.org/10.1145/3469029

[23] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F Sweeney. 2009. Producing wrong data without doing anything obviously wrong! *ACM Sigplan Notices* 44, 3 (2009), 265–276.

[24] Wei Niu, Xiaolong Ma, Sheng Lin, Shihao Wang, Xuehai Qian, Xue Lin, Yanzhi Wang, and Bin Ren. 2020. PatDNN: Achieving Real-Time DNN Execution on Mobile Devices with Pattern-Based Weight Pruning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 907–922. https://doi.org/10.1145/3373376.3378534

[25] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. https://doi.org/10.1109/ISCA45697.2020.00045

[26] Hao Shen, Ying Tan, Jun Lu, Qing Wu, and Qinru Qiu. 2013. Achieving Autonomous Power Management Using Reinforcement Learning. *ACM Trans. Des. Autom. Electron. Syst.* 18, 2, Article 24 (apr 2013), 32 pages. https://doi.org/10.1145/2442087.2442095

[27] Richard S Sutton. 1988. Learning to predict by the methods of temporal differences. *Machine learning* 3, 1 (1988), 9–44.

[28] The Jetson Platform Team. 2022. *NVIDIA Jetson TX2 Specifications*. NVIDIA. Retrieved March, 2022 from https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-tx2/

[29] The Jetson Platform Team. 2022. *NVIDIA Jetson TX2 Thermal Design Guide*. NVIDIA. Retrieved March, 2022 from https://developer.nvidia.com/embedded/dlc/jetson-tx2-thermal-design-guide

[30] Michel Tokic and Günther Palm. 2011. Value-difference based exploration: adaptive control between epsilon-greedy and softmax. In *Annual conference on artificial intelligence*. Springer, 335–346.

[31] Chengcheng Wan, Muhammad Santriaji, Eri Rogers, Henry Hoffmann, Michael Maire, and Shan Lu. 2020. ALERT: Accurate learning for energy and timeliness. In *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*. 353–369.

[32] Christopher JCH Watkins and Peter Dayan. 1992. Q-learning. *Machine learning* 8, 3 (1992), 279–292.

[33] Carole-Jean Wu, David Brooks, Kevin Chen, Douglas Chen, Sy Choudhury, Marat Dukhan, Kim Hazelwood, Eldad Isaac, Yangqing Jia, Bill Jia, Tommer Leyvand, Hao Lu, Yang Lu, Lin Qiao, Brandon Reagen, Joe Spisak, Fei Sun, Andrew Tulloch, Peter Vajda, Xiaodong Wang, Yanghan Wang, Bram Wasti, Yiming Wu, Ran Xian, Sungjoo Yoo, and Peizhao Zhang. 2019. Machine Learning at Facebook: Understanding Inference at the Edge. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 331–344. https://doi.org/10.1109/HPCA.2019.00048